Chapter 1: The Software Problem

Presented By Shagoofi Anjum

What is the Software Problem?

- Software engineering deals with **industrial-strength software**, not small student programs.
- Student programs are made for **demonstration**, while industrial software is used in **real business operations**.
- Failure of industrial software can cause:
  - Financial loss
  - Business disruption
  - Property damage
  - Loss of life (in critical systems)

Student Software vs Industrial-Strength Software

- **Student Software**
- Short-term use
- Bugs acceptable
- No real dependency
- Low documentation
- Built quickly
- **Industrial-Strength Software**
- Long-term use
- Real users depend on it
- Bugs unacceptable
- Requires reliability, usability, maintainability
- Cost is much higher
- ☞ Industrial software cost ≈ **10 times** student software.

Three Driving Forces of Software Projects

- Industrial software projects are driven by **three main forces**:
- **Cost**
- **Schedule**
- **Quality**
- All three must be balanced.

Software Cost

- Software development is **labor-intensive.**
- Cost is measured in **person-months.**
- Size of software is measured in:
  - LOC (Lines of Code)
  - KLOC (Thousands of Lines of Code)
- **Productivity**
- Productivity $= \frac{LOC\ delivered}{Person\text{–}Months}$
- Typical productivity: **few hundred to ~1000 LOC/person-month**
- Software is expensive because human effort is expensive.

Software Schedule

- Schedule = **time required to deliver software**
- Late delivery is common in software projects.
- Market pressure demands **shorter development cycles**.
- Improving productivity helps:
  - Reduce cost
  - Reduce development time

Software Quality

- Software quality means software should:
- Do what it is supposed to do
- Do it correctly
- Do it consistently

Main Software Quality Attributes (As per international standards)

- Functionality
- Reliability
- Usability
- Efficiency
- Maintainability
- Portability

Reliability (Most Important Attribute)

- Reliability = **failure-free operation**
- Software failures happen due to **defects (bugs)**.
- Quality is often measured as:
- Defects *per KLOC*
- Best practices aim for **< 1 defect per KLOC**.

Software Maintenance

- Software needs maintenance because:
  - Defects remain after delivery
  - User requirements change
  - Environment changes
- **Important Fact**
- Maintenance cost is very high.
- Maintenance may consume **60-80%** of total software cost.

Scale (Size of Software)

- Industrial software is **large and complex**.
- Methods used for small programs **do not scale**.
- Large software requires:
  - Formal engineering
  - Strong project management

Change (Key Characteristic)

- Requirements are **never fully known at start.**
- Changes occur:
  - During development
  - After deployment
- Handling change poorly can consume **30–40% extra cost.**
- Software engineering methods must **handle change efficiently.**

Chapter 2: Software Processes

Software Engineering (Process View)

- **Definition:**
  Software Engineering is a **systematic approach** to the **development, operation, maintenance, and retirement** of software.
- **Key Idea:**
- Goal is **high Quality & Productivity (Q&P)**
- Achieved using **people, process, and technology**
- **Process is the core element**

People, Process, and Technology

- **People** → Skills, experience, teamwork
- **Process** → Defines *what to do* and *how to do*
- **Technology** → Tools that support the process
  - Process helps people achieve **higher quality with less effort.**

What is a Process?

A **process** is a **sequence of steps or activities** performed to achieve a specific goal.

- **In Software Engineering:**
- Goal → Deliver working, high-quality software
- Process defines:
  - Activities
  - Order of execution
  - Constraints

Software Project

A **software project** is one **instance of executing a software process** to develop a software system for a specific client or user.

- **Important Point:**
  - Same process → multiple projects
  - Each project is unique

Process Vs Project

Process	Project
General framework	One execution
Reusable	Temporary
Defines activities	Performs activities
Organization-wide	Client-specific

Process Specification vs Actual Process

- **Process Specification** → What is *planned*
- **Actual Process** → What is *actually followed*

Process Model

- A **process model** is a **general framework or guideline** that defines:
  - Stages of development
  - Order of stages
  - Best practices
- **Purpose:**
  - Reduce cost
  - Improve quality
  - Reduce development time

Component Software Processes

- Software process is divided into **four major component processes**:

  1. Development Process
  - Engineering activities
  - Requirements, design, coding, testing

  2. Project Management Process
  - Planning
  - Scheduling
  - Monitoring & control

  3. Configuration Management Process
  - Version control
  - Change management
  - Product integrity

  4. Process Management Process
  - Improves software process itself
  - Analyzes past projects
  - Introduces best practices

Software Development Process

The development process defines **technical activities** required to **produce the software product**.

- **Focus:**
  - Engineering
  - Quality control
  - Core of software process

Project Management Process

This process ensures the project is completed:

- Within **budget**
- Within **schedule**
- With **required quality**

- **Includes:**
  - Planning
  - Risk management
  - Tracking & control

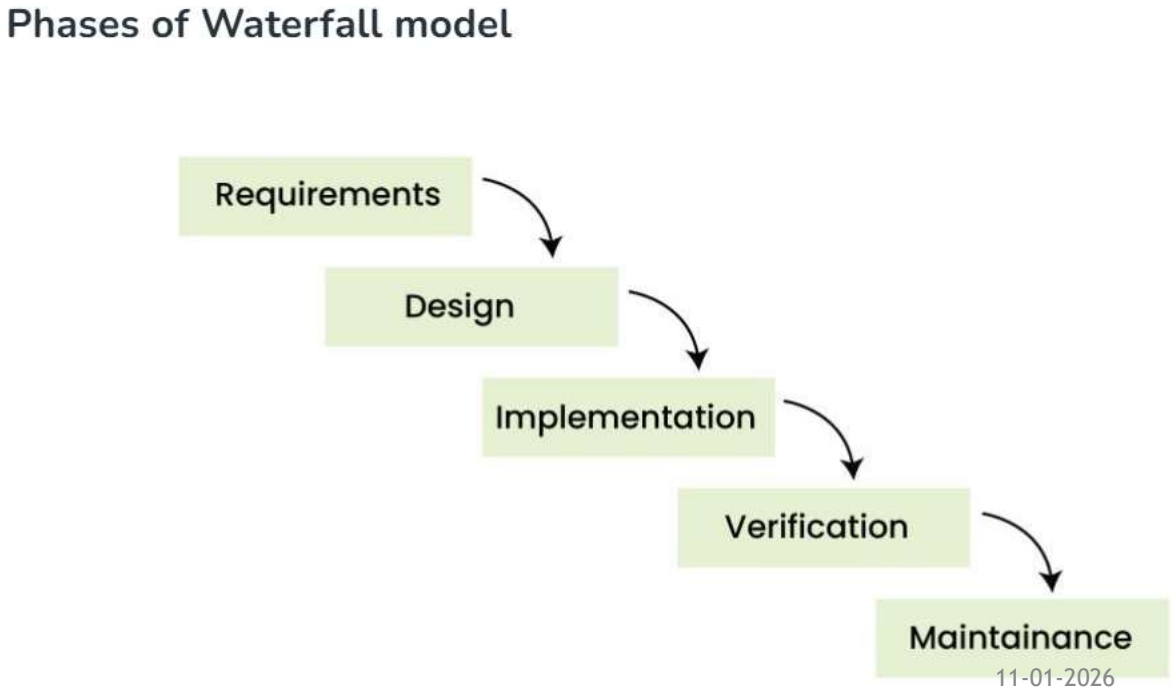
Software Development Process Models

Process models describe **how software development activities are organized and executed.**

Software Process Model

Waterfall Model

The Waterfall Model is a **linear and sequential software development model** in which each phase must be completed before the next phase begins.

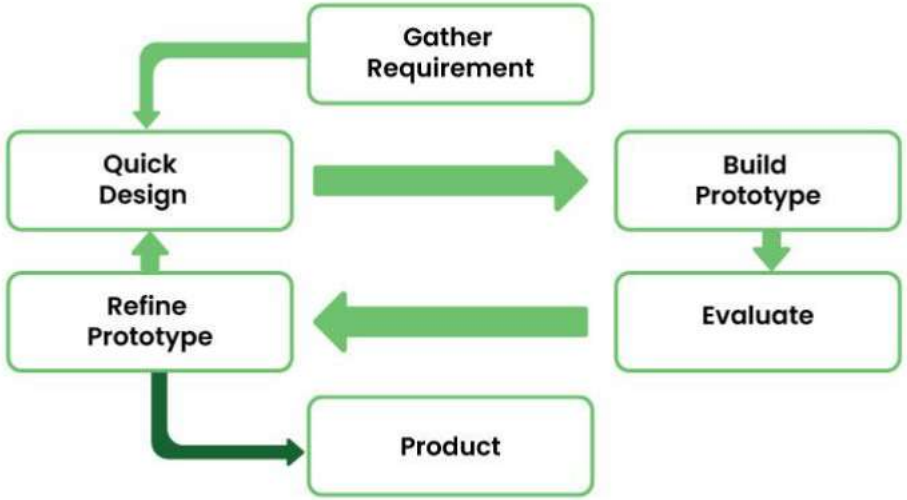


- **Advantages**
  - Simple and easy to understand
  - Clear documentation
  - Easy to manage
  - Suitable for projects with stable requirements
- **Disadvantages**
  - Cannot handle changing requirements
  - Working software is available very late
  - High risk for large projects
  - Rigid structure

Prototyping Model

The Prototyping Model involves building a **temporary working prototype** to understand unclear requirements before developing the final system.

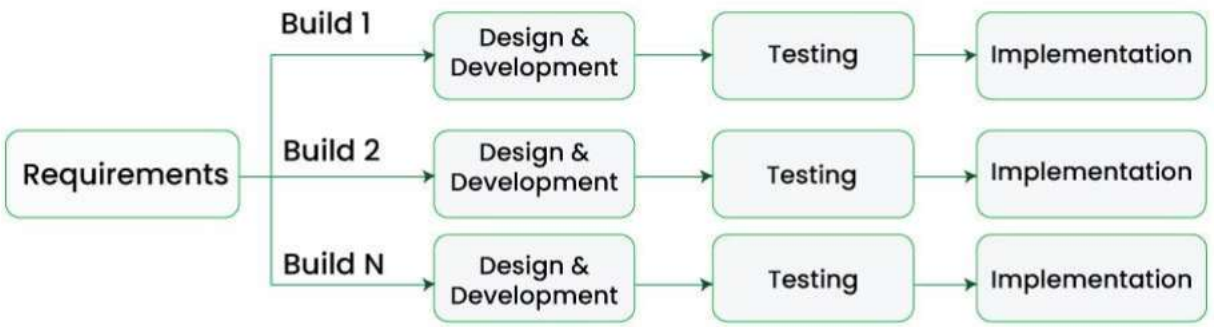
Phases of Prototype Model:



- **Advantages**
  - Better requirement understanding
  - High user involvement
  - Reduces requirement risk
  - Improves user satisfaction
- **Disadvantages**
  - Extra development cost
  - Poor design if prototype becomes final system
  - User may confuse prototype with final product

Incremental Model

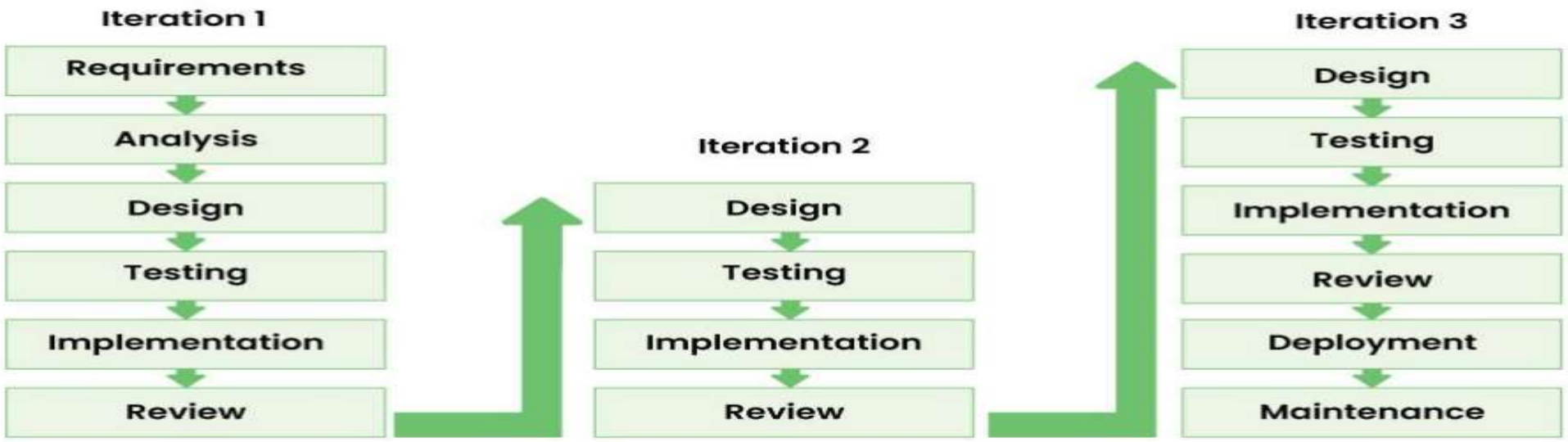
- In the Incremental Model, the software is developed and delivered in **small functional increments**, with each increment adding new features.
- Phase Of **Incremental Model**



- **Advantages**
  - Early delivery of working software
  - Easy testing and debugging
  - Flexible to changes
  - Reduced project risk
- **Disadvantages**
  - Proper planning required
  - Integration problems may occur
  - Overall cost may be higher

Iterative Model

- The Iterative Model develops software through **repeated cycles (iterations)**, improving the system with each iteration.
- Phases Of **Iterative Model**

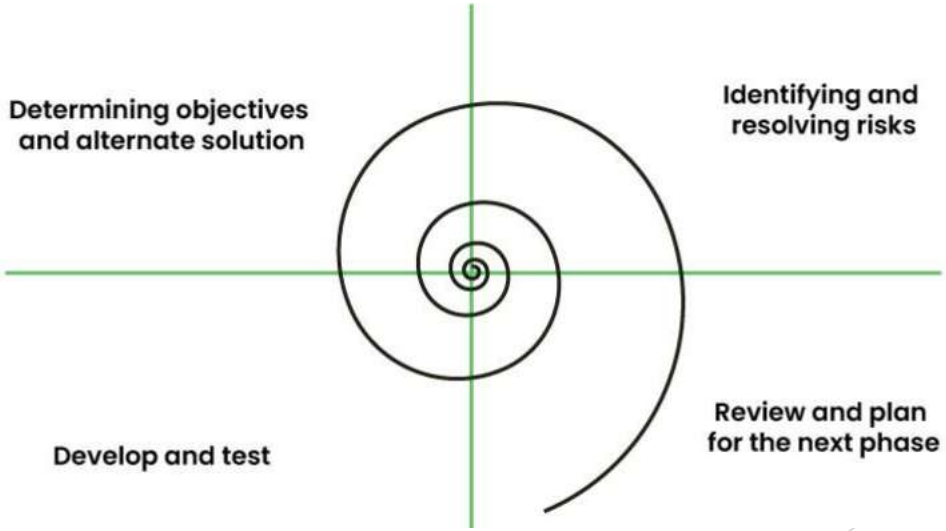


- **Advantages**
  - Handles changing requirements
  - Early feedback from users
  - Improved quality over iterations
- **Disadvantages**
  - Management complexity
  - Requires continuous user involvement
  - Less documentation

Spiral Model

The Spiral Model is a **risk-driven model** that combines **Waterfall and Prototyping**, focusing on risk analysis.

Phases Of **Spiral Model**

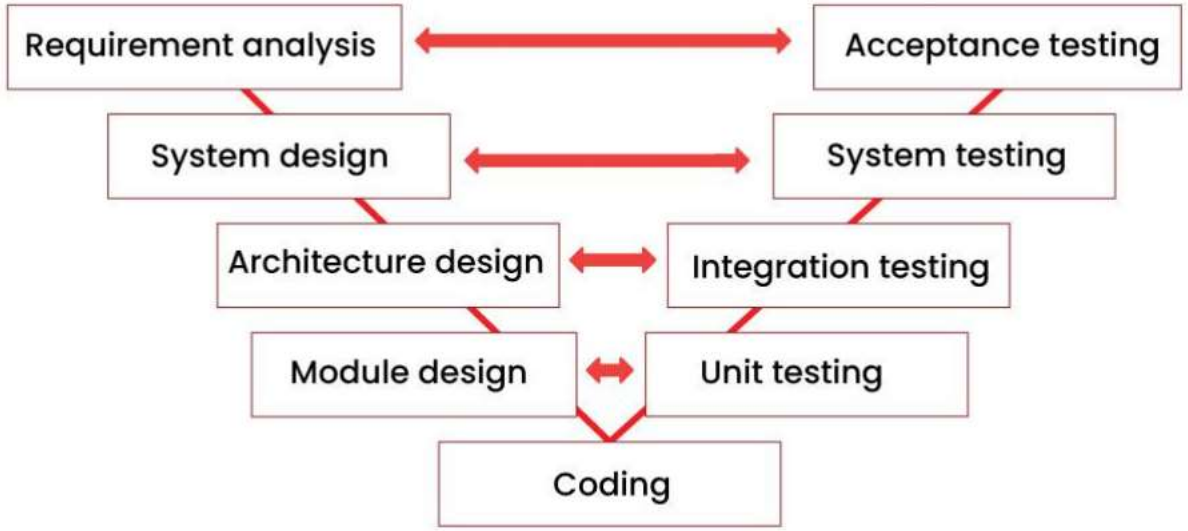


- **Advantages**
  - Best for high-risk projects
  - Early identification of risks
  - Flexible and iterative
  - Continuous customer feedback
- **Disadvantages**
  - Complex model
  - Expensive
  - Not suitable for small projects
  - Requires risk analysis expertise

V-Model (Verification and Validation Model)

The V-Model is an extension of the Waterfall Model where **each development phase has a corresponding testing phase.**

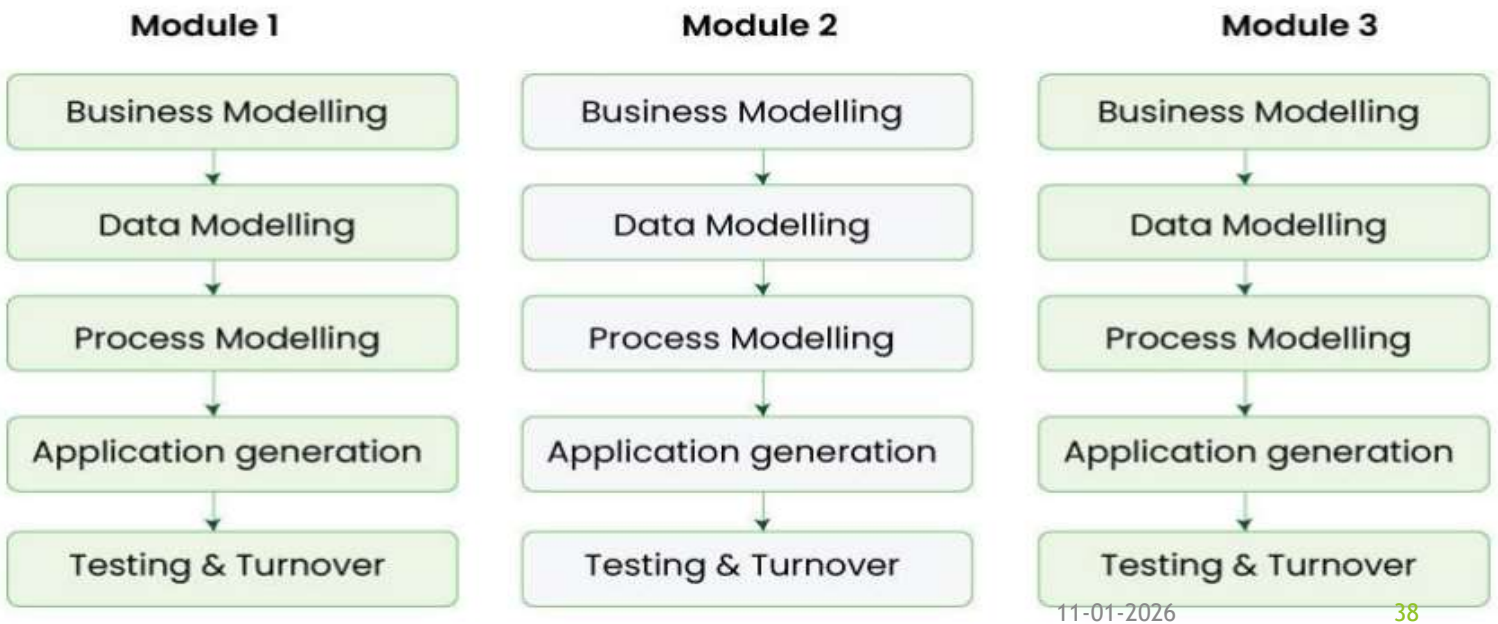
Phases of V-model



- **Advantages**
  - Early testing
  - High quality assurance
  - Clear deliverables
  - Simple and systematic
- **Disadvantages**
  - Rigid model
  - Requirement changes are difficult
  - No early working software

RAD Model (Rapid Application Development)

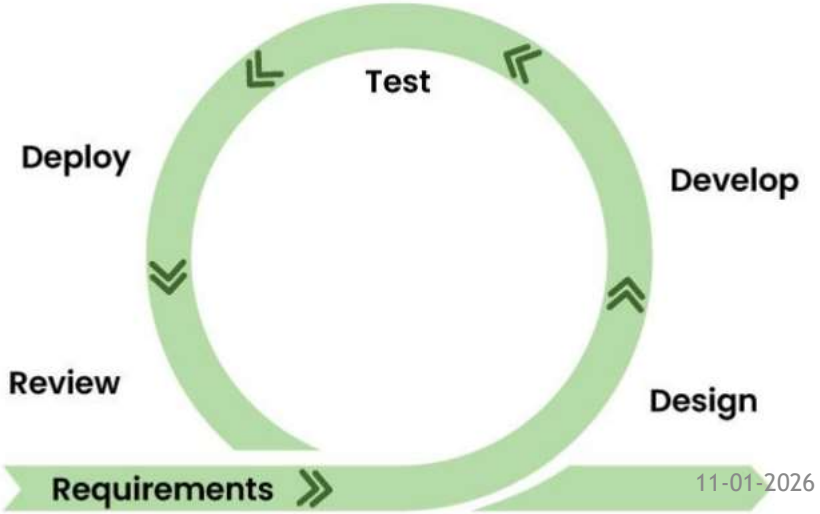
- The RAD Model focuses on **fast development** using reusable components and automated tools.
- Phases of RAD



- **Advantages**
  - Very fast development
  - High productivity
  - Reusability of components
  - Continuous user involvement
- **Disadvantages**
  - Skilled developers required
  - Not suitable for large or complex systems
  - High dependency on tools

Agile Model

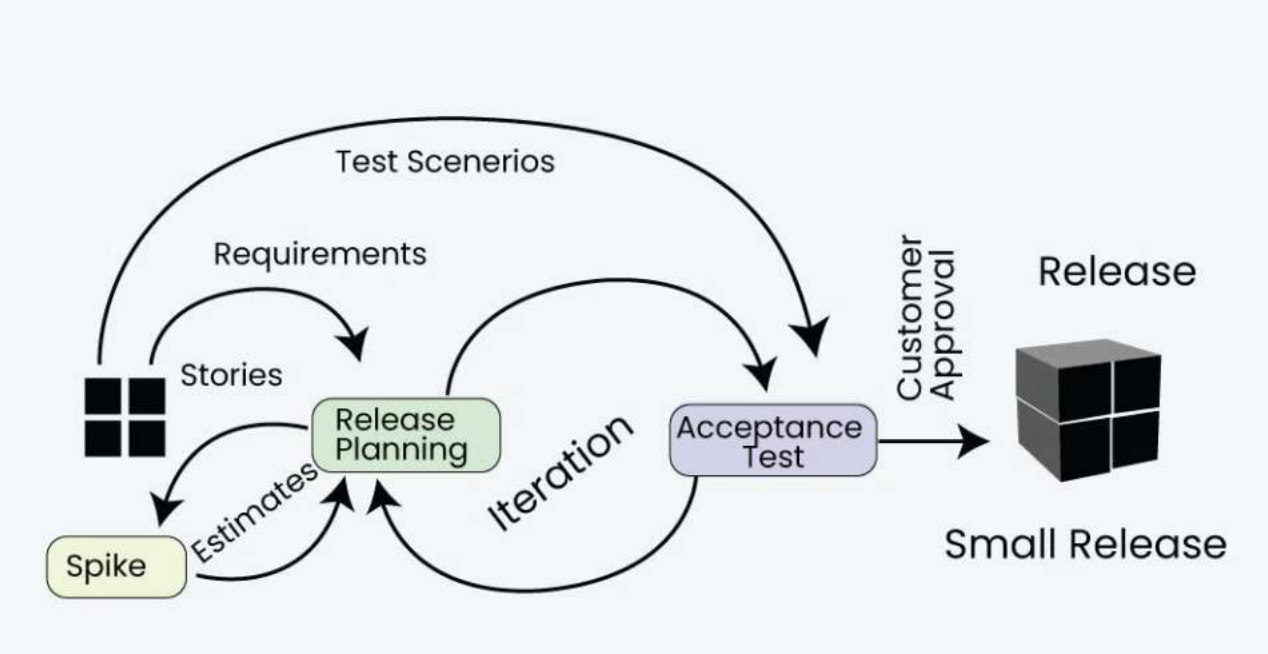
- The Agile Model is an **iterative and incremental approach** that develops software in **small time-boxed iterations called sprints.**
- The Agile Model was created mainly to make changes in the middle of **software development** so that the software project can be completed quickly.
- **Phases**



- **Advantages**
  - Highly flexible
  - Fast delivery
  - Customer satisfaction
  - Easily handles change
- **Disadvantages**
  - Less documentation
  - Difficult for large teams
  - Requires experienced developers
  - Scope creep risk

Extreme Programming (XP)

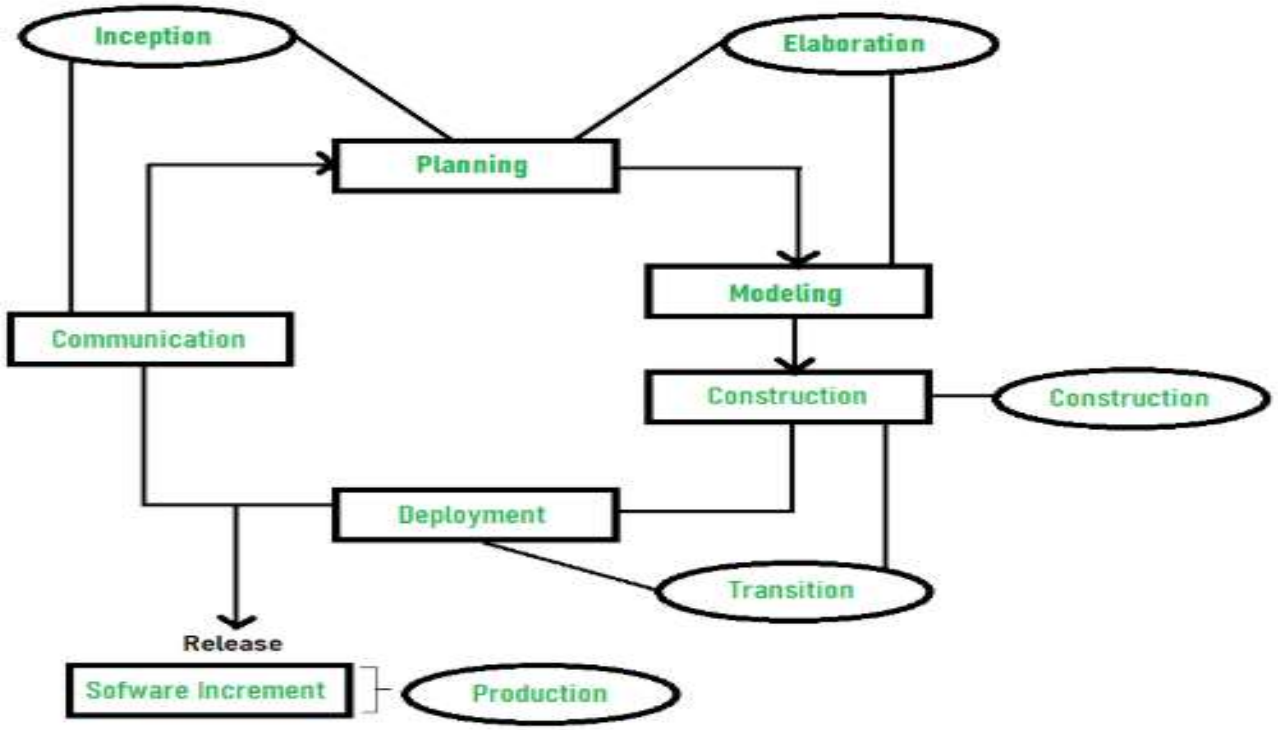
- Extreme Programming is an Agile methodology that emphasizes **continuous testing, pair programming, and customer involvement**.
- **Phases**



- **Advantages**
  - High-quality code
  - Continuous testing
  - Quick response to change
- **Disadvantages**
  - Requires strict discipline
  - Not suitable for large projects
  - Minimal documentation

Rational Unified Process (RUP)

- RUP is a **structured, iterative, object-oriented process model** used for large and complex projects.
- Phases



- **Advantages**
  - Well-documented
  - Risk-focused
  - Scalable
  - Industry-proven
- **Disadvantages**
  - Complex and heavy
  - Expensive to implement
  - Requires trained professionals